

¿Quién juzga al juez?

La semana pasada llegamos a algo incómodo: ninguna métrica, por sí sola, alcanza para decir si un sistema es bueno. Hoy sumamos un juez —otro modelo que califica— y la mirada humana, y con eso armamos el harness de tres dimensiones que entregan como M2.

LLM-AS-A-JUDGE

SESGOS DEL JUEZ

SU HARNESS = M2

La sesión de hoy.



Recap: por qué una métrica no basta



LLM-as-a-judge: rúbrica y setup  Lab A



Los sesgos del juez y cómo mitigarlos  Lab B



Evaluación humana e inter-rater agreement



Datasets sintéticos y adversariales • red-teaming

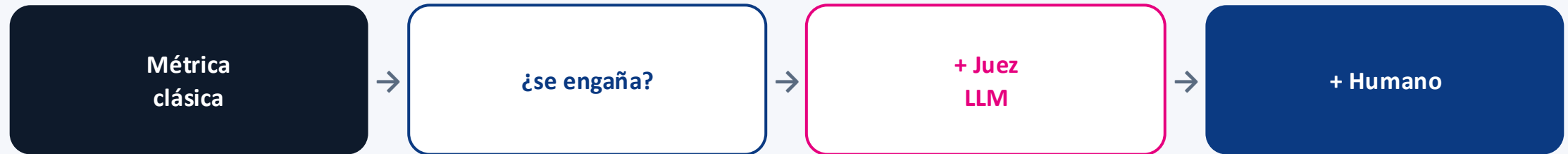


El harness de 3 dimensiones + scorecard  Lab C • entrega M2

Medir no basta. Hay que juzgar.

Por qué necesitamos un juez para lo que ninguna métrica automática captura.

De dónde venimos.



La semana pasada lo vimos en el lab: **BLEU —que solo cuenta cuántas palabras coinciden— llegó a premiar la respuesta equivocada**, y la perplexity nos dice si el texto suena fluido, no si es verdad.

Por eso hoy sumamos dos miradas que sí **juzgan la calidad**: un juez que es otro modelo (escala como una máquina) y la persona (que sigue siendo el patrón oro).

No podemos leer **500 respuestas** a mano.

A MANO

El patrón oro

Evaluar leyendo cada respuesta es lo más confiable, pero no escala a cientos de casos... ¿y los costos?

UNA MÉTRICA

Escala, no entiende

Una métrica automática corre en segundos, pero no entiende si la respuesta es correcta o útil.

LLM-AS-A-JUDGE

Lo mejor de ambos

Juicio parecido al humano, a escala de máquina: un LLM lee y califica cada respuesta.

Pero ¡cuidadito! el juez hereda sesgos — los cazamos en el bloque siguiente.

¡Pero!

Como cualquier modelo de IA, el juez hereda sesgos.



Un modelo califica a otro.

Cómo montar un juez con rúbrica, y en qué modos puede trabajar.

Un LLM con rúbrica califica.

Se le da a un LLM la pregunta, la respuesta que queremos evaluar y una rúbrica de qué cuenta como buena respuesta. El juez devuelve un puntaje —y, si se lo pedimos, una justificación en texto de por qué lo dio.

Potente: entiende significado y matices donde una métrica de palabras se queda corta.

Peligroso: tiene sesgos, y hay que domarlo antes de confiar en su veredicto — eso es el bloque siguiente.

Construyamos un juez, juntos.

EL CASO

Un sistema que, a partir de tu perfil de LinkedIn (y otras redes) y unas vacantes objetivo, arma un CV ajustado a tu experiencia y al rol. Escribimos una primera versión y la miramos.

SU TAREA AHORA

En una línea: ¿qué hace “bueno” a ese CV para ese rol? Eso es la rúbrica del juez — y es la parte difícil, no el código.

Antes de la teoría: **escriban su rúbrica** y prueben a calificar el CV a mano. Después comparamos con lo que “debía” llevar.

Qué lleva un buen prompt de juez (y dónde acertamos).

CRITERIO CLARO

Definan qué es una buena respuesta en su dominio; sin criterio, el juez inventa el suyo.

ESCALA DEFINIDA

Un rango 1–5 con anclas por nivel, no “del 1 al 10 a ojo”. Cada nivel debe significar algo.

FORMATO DE SALIDA

Pidan solo un número (o un JSON) para poder parsear el veredicto y automatizarlo.

REFERENCIA (OPCIONAL)

Si tienen la respuesta gold, dénsela al juez para que compare contra ella.

El juez puede trabajar de tres formas.

MODO	CÓMO	CUÁNDO USARLO
 Pointwise	le pone una nota a UNA respuesta (1–5)	calificar cada respuesta del baseline → el scorecard de M2
 Pairwise	compara dos respuestas y elige la mejor	¿mi cambio sirvió? versión A (antes) vs B (después)
 Reference-based	compara contra una respuesta gold	cuando el eval set trae la salida esperada

¿Hay más de tres? sí — un panel de varios jueces, G-Eval (un juez que razona paso a paso), rúbricas multi-criterio... Estos tres cubren el 90%. El harness de M2 usa pointwise.

Al notebook — Lab A

Monten su juez con rúbrica

Escriben un prompt de juez con una rúbrica 1–5 para su dominio, se lo pasan a un LLM pequeño y obtienen un puntaje por respuesta. Comparan una respuesta claramente buena contra una pobre y ven si el juez las distingue.

```
prompt con rúbrica → LLM juez → score 1-5 → S06_Lab_Harness_de_evaluacion.ipynb
```

Qué aprendimos montándolo.

- **Acertó lo grueso** con una rúbrica clara, el juez separó un CV sólido de uno flojo — funciona.
- **Se le coló un sesgo** tendió a premiar el CV más largo, aunque no era mejor. Guarden esa pista.
- **El prompt es el producto** cambiar la rúbrica cambió la nota: el juez es código, se versiona y se prueba.
- **Confiamos sin verificar** ¿y si ese sesgo estaba decidiendo por nosotros? Eso es exactamente lo siguiente.

El juez también se equivoca.

Posición, longitud y auto-preferencia: los tres vicios del juez, y cómo domarlos.

El juez tiene tres vicios conocidos.

POSICIÓN



Prefiere el orden

Prefiere la 1ª (o la 2ª) respuesta por dónde aparece, no por su calidad.

Ejemplo: al invertir el orden de dos respuestas, el juez cambia de favorito.

Zheng et al. 2023 • Wang et al. 2023

LONGITUD



Premia lo largo

Da mejor nota a respuestas largas aunque no sean mejores — más texto no es más calidad.

Ejemplo: una respuesta más larga, pero igual de correcta, recibe mejor nota.

Dubois et al. 2024 • Zheng et al. 2023

AUTO-PREFERENCIA



Se prefiere a sí mismo

Prefiere texto de su propia familia de modelo: no es neutral consigo mismo.

Ejemplo: un juez GPT-4 prefiere respuestas de GPT-4 sobre otras igual de buenas.

Panickssery et al. 2024

Documentados y medibles – por eso mismo, mitigables (bloque siguiente).

Cada sesgo tiene una consecuencia real.

SESGO	SE VE ASÍ	CONSECUENCIA
 Posición	cambia el veredicto al invertir el orden $A \leftrightarrow B$	comparaciones no confiables
 Longitud	la respuesta más larga gana casi siempre	infla sistemas verbosos
 Auto-preferencia	un GPT-juez prefiere respuestas de GPT	evaluación no neutral

Lo bueno: estos sesgos son medibles y, por lo tanto, mitigables.

¿Cómo los mitigamos?



POSICIÓN

...



LONGITUD

...



AUTO-PREFERENCIA

...

Lluvia de ideas del grupo – lo llenamos aquí y en la siguiente lo comparamos.

¿Cómo los mitigamos? Domándolo.

INTERCAMBIAR POSICIÓN

Evalúa A-B y B-A, y promedia. Si el veredicto cambia, era sesgo, no calidad.

CONTROLAR LONGITUD

Pide concisión en la rúbrica y compara respuestas de longitud similar.

RÚBRICA EXPLÍCITA

Ancla cada nivel 1–5 con una descripción para que no juzgue “a ojo”.

CALIBRAR CON HUMANOS

Revisa a mano una muestra y mide si el juez concuerda con las personas.

Al notebook — Lab B

Cacen el sesgo de posición

Toman dos respuestas —una claramente mejor— y le piden al juez que elija: primero en un orden, luego con el orden invertido. Ven si el veredicto se voltea. Después aplican la mitigación: promediar los dos órdenes.

juez pairwise A-B vs B-A → ¿se voltea?

Qué probaron.

- **La prueba del sesgo** presentar el mismo par en los dos órdenes (A-B y B-A) es como se caza el sesgo de posición.
- **Puede no voltearse** con un par muy desigual, hasta un juez pequeño sigue la calidad; el sesgo asoma cuando las dos respuestas están parejas.
- **Pero ya tienen el detector** evaluar en ambos órdenes y promediar: el día que se voltee, lo van a cazar.

El patrón oro sigue siendo humano.

Cuándo vale el costo, y cómo saber si dos personas concuerdan.

Cuando de verdad importa (mucho, mucho), pregúntenle a una persona.

El juicio humano con rúbrica sigue siendo la referencia contra la cual se calibra todo lo demás —el LLM-juez incluido. Su costo es tiempo y consistencia: las personas se cansan y no siempre concuerdan entre sí.

La regla práctica: **usen humano** para calibrar el juez y para los casos de alto riesgo; usen el **LLM-juez** para escalar el resto. Así lo usan hasta las IAs agénticas (como Claude Code): escalan a una persona los casos difíciles.

¿Dos personas califican igual?

Si dos anotadores no concuerdan, el problema es la rúbrica ambigua, no el modelo. Se mide el acuerdo con Cohen's kappa, que corrige el acierto por azar: por encima de 0.6 es aceptable, por encima de 0.8 es fuerte.

Baja concordancia ⇒ **arreglen la rúbrica** antes de confiar en cualquier número, venga de un humano o de un LLM.
Un juez calibrado sobre una rúbrica ambigua no mide nada.

Rompan su propio sistema.

Generar casos difíciles a propósito, antes de que lo haga el usuario.

Fabriquen los casos difíciles.

Además de los ejemplos gold reales, pueden generar —incluso con un LLM— casos de borde a propósito: preguntas ambiguas, con trampa, o fuera de dominio. Eso sí: valídenlos a mano antes de meterlos al eval set.

Un eval set solo con casos fáciles miente: todo se ve bien hasta que llega el usuario real. Los bordes son donde se ve la calidad de verdad — y donde su sistema se va a romper primero.

Ataquen su sistema antes que el usuario.

Tres pruebas que deberían hacer SIEMPRE.

ALUCINACIÓN



¿Inventa o admite?

Pregúntenle algo cuya respuesta no existe: ¿se lo inventa con seguridad, o admite que no sabe?

FUERA DE DOMINIO



¿Pone límites?

Algo que su sistema NO debería responder: ¿reconoce el límite o contesta igual?

SEGURIDAD



¿Se niega bien?

Entradas tóxicas o con trampa: ¿se niega de forma clara, o cae en la provocación?

Inclúyanlos: al menos 2 casos adversariales en el eval set de M2.

Todo junto: su harness.

Las tres dimensiones, ejecutables, produciendo el scorecard del baseline.

Tres dimensiones, un solo harness.

1 • MÉTRICA CLÁSICA



Automática y barata

Similitud por embeddings o ROUGE.
Rápida y reproducible, con sus sesgos conocidos.

2 • LLM-AS-A-JUDGE



Un LLM califica

Con rúbrica y control de sesgos
(intercambiar posición, controlar longitud).

3 • EVAL SET DE DOMINIO



Sus ejemplos

Sus 10 ejemplos gold más al menos 2 casos adversariales. La medida que sí importa.

El harness corre las tres sobre su modelo de M1 y arma un scorecard del baseline.

Qué es un scorecard.

Una tabla con el puntaje del baseline por dimensión sobre su eval set, más una lectura honesta de dónde falla. No es un número mágico: es un retrato del sistema hoy.

DIMENSIÓN	PUNTAJE BASELINE
● Similitud por embeddings	0.71
● LLM-juez (1-5)	3.4
● Aciertos de dominio	6 / 10

Este scorecard ES la entrega M2.

Al notebook — Lab C

Armen el harness y el scorecard

Juntan las 3 dimensiones en una función que recibe su eval set y su modelo, y produce el scorecard del baseline. Lo guardan en el repo. Esto es el corazón de M2.

```
harness(eval_set, modelo) → scorecard.csv • entrega M2
```

Resumen: lo que produjeron.

- **Un harness reutilizable** la misma función evaluará su RAG (M3), su parte visual (M4) y producción (M5).
- **Un scorecard del baseline** tres dimensiones, una lectura honesta de dónde falla hoy.
- **La vara del semestre** toda mejora futura se mide contra este número.

Lo que entregan (M2 • 10%).

1

El harness ejecutable: 3 dimensiones sobre su eval set (≥ 10 gold + ≥ 2 adversariales).

2

El scorecard del baseline + un párrafo de lectura honesta de dónde falla.

3

README con la rúbrica del juez y una frase en lenguaje sencillo de qué evalúan.

Repo de GitHub a EAFIT Interactiva. La rúbrica completa está en el documento de la asignación.

Ya no preguntan “¿es bueno?”.
Ahora lo miden con su propio harness.

Cierra el Módulo 2. El harness que hicieron hoy es la vara con la que medirán RAG, visión y producción el resto del semestre.